Reviewer Pack — Minimal Rank of Quotient Algebras over XOR-AND Tree Structur…

Entry bb81e51d90d8 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 13:22:52 UTC

Minimal Rank of Quotient Algebras over XOR-AND Tree Structures

Entry ID: `bb81e51d90d8` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-26 13:22:52 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Quotient Algebras) **Field B** (complexity object): Complexity Theory: XOR-AND Trees

Statement:

> ["For any AC0 circuit C with n inputs computing an XOR-AND tree function f, the minimal rank of the quotient algebra Q = K[x_1,...,x_n]/I(C), where I(C) is the ideal generated by the polynomials corresponding to C's gates, is at least Ω(log n).", 'Equivalently, for any AC0 circuit C with n inputs computing an XOR-AND tree function f, there exists a polynomial g ∈ Q such that the degree of g is at least Ω(log n).']

Rationale (proposer's reasoning):

> ['Quotient algebras provide a way to study algebraic properties of functions. If this conjecture holds, it would imply that certain algebraic properties are inherent in the structure of XOR-AND trees, which could lead to new insights into complexity theory.', 'XOR-AND trees are known to be hard for AC0 circuits, and if there is a direct relationship between their structure and the algebraic properties of quotient algebras, it could provide a new angle for proving lower bounds on the size or depth of AC0 circuits.']

Taxonomy category: `AC0_PARITY` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `3ff9076f9aa8e373`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all 30 random seeds, the average degree of any polynomial g in Q is at least Ω(log n) and no seed produces a polynomial with degree less than Ω(log n). The conjecture is falsified if there exists a seed where the average degree of polynomials in Q is less than Ω(log n) or where the degree of a polynomial g ∈ Q is less than Ω(log n).

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "quotient algebra" AND "XOR-AND trees" AND minimal rank" - "algebraic geometry" AND "AC0 circuit" AND "degree Ω(log n)" - "polynomial ideal" IN (arXiv.org, arxiv.org/abs) AND "XOR-AND tree structures"

Top relevant hits considered: - [http://arxiv.org/abs/2401.14517v3] Efficient List-decoding of Polynomial Ideal Codes with Optimal List Size - [http://arxiv.org/abs/0706.1952v1] Canonical representatives for residue classes of a polynomial ideal and orthogonality - [http://arxiv.org/abs/alg-geom/9610006v1] Bounds for the Hilbert function of polynomial ideals and for the degrees in the Nullstellensatz

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def generate_xor_and_tree(n):
    if n == 1:
        return 'x1'
    else:
        left = generate_xor_and_tree(n // 2)
        right = generate_xor_and_tree(n - n // 2)
        return f'({left} & {right}) | ({left} ^ {right})'

def ac0_circuit_to_polynomial(circuit):
    if circuit == 'x1':
        return 'x1'
    elif circuit.startswith('(') and circuit.endswith(')'):
        left, op, right = circuit[1:-1].split()
        if op == '&':
            return f'({ac0_circuit_to_polynomial(left)} & {ac0_circuit_to_polynomial(right)})'
        elif op == '^':
            return f'({ac0_circuit_to_polynomial(left)} ^ {ac0_circuit_to_polynomial(right)})'
    else:
        raise ValueError("Invalid circuit format")

def run_trial(seed: int) -> dict:
```

```
    random.seed(seed)
    n = 30
    if n < 5 or n > 40:
        return {
            "metric_name": "degree",
            "metric_value": -1,
            "instances_tested": 0,
            "conjecture_holds": False,
            "counterexample": "invalid_n"
        }

    degree_sum = 0
    instances_tested = 0

    for _ in range(30):
        circuit = generate_xor_and_tree(n)
        polynomial = ac0_circuit_to_polynomial(circuit)

        # Count the number of XOR and AND operations
        xor_count = polynomial.count('^')
        and_count = polynomial.count('&')

        # The degree is the maximum of XOR and AND counts
        degree = max(xor_count, and_count)

        if degree < math.log(n):
            return {
                "metric_name": "degree",
                "metric_value": -1,
                "instances_tested": 0,
                "conjecture_holds": False,
                "counterexample": f"seed={seed}, n={n}, degree={degree}, expected=Ω(log {n})"
            }

        degree_sum += degree
        instances_tested += 1

    mean_degree = degree_sum / instances_tested
    return {
        "metric_name": "degree",
        "metric_value": mean_degree,
        "instances_tested": instances_tested,
        "conjecture_holds": mean_degree >= math.log(n),
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {{'seed': {seed}, **trial_result}}")
        results.append(trial_result)
```

```
    mean_degree = sum(result["metric_value"] for result in results if result["instances_tested"] >
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_degree} std=NA support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample='degree < Ω(log n)' first_failing_seed={first_fai
    else:
        print("RESULT: INCONCLUSIVE")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9dbebe98.py", line 91, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9dbebe98.py", line 55, in run_trial
    polynomial = ac0_circuit_to_polynomial(circuit)
                 ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9dbebe98.py", line 30, in ac0_circuit_t
    left, op, right = circuit[1:-1].split()
    ^^^^^^^^^^^^^^^
ValueError: too many values to unpack (expected 3)
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which means we cannot verify the conjecture's support or falsification based on the pre-registered criteria. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	16504
2	preregistration	ollama_remote	glm4:latest	0	0	6072
3	novelty	ollama_remote	glm4:latest	0	0	4857

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
4	novelty	ollama_remote	glm4:latest	0	0	6569
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14548
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7816
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	28210
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12512
9	judge	ollama_remote	glm4:latest	0	0	48752

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 145840 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/bb81e51d90d8.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/bb81e51d90d8.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/bb81e51d90d8.tar.gz` (if generated)